
Empirical Estimation Models

Empirical estimation models are mathematical models that estimate the effort, cost and development time of a software project based on historical project data and experience.

INTRODUCTION

- Estimation models uses **empirically derived formulas** to predict the estimates.
- Here we **conduct a study** on some completed projects.
- From those observation we form some **statistical formulas**.
- We can use this formulas to **estimate the cost** of other projects.
- The structure of empirical estimation models is a formula, derived from data collected from **past software projects**.

THE STRUCTURE OF ESTIMATION MODELS

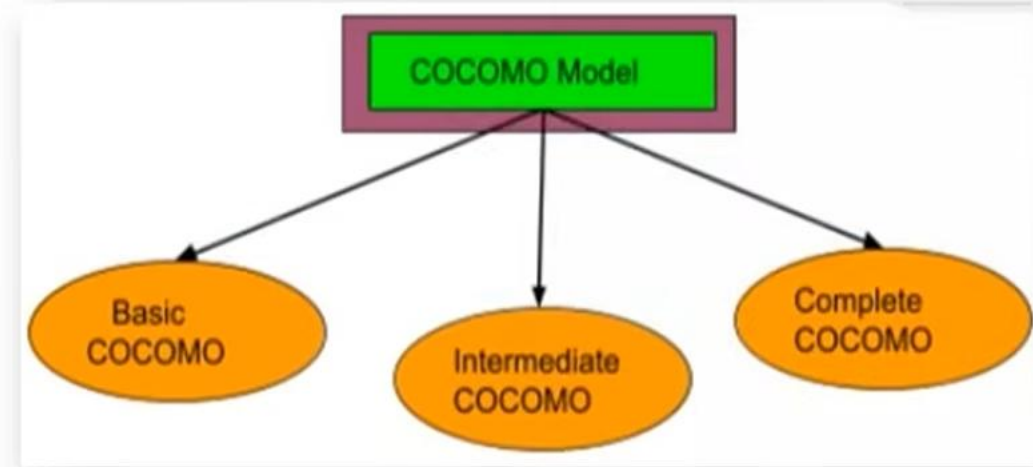
- The overall structure of such models takes the form –

$$E = A + B * (e_v)^C$$

- Where,
 - A, B and C – empirically derived constants
 - E – effort in person-months
 - e_v – estimation variable (LOC or FP)

About COCOMO Model

- It was developed by a scientist Barry Boehm in 1981.
- The COCOMO (**Constructive Cost Model**) is one of the most popularly used software cost estimation models.
- This model depends on the size means number of lines of code for software product development.
- It estimates Effort required for project, Total project cost & Scheduled time of project.



Software Project Types

In COCOMO, projects are categorized into three types:

1. Organic Type:

- Project is small and simple. (2-50 KLOC)
- Few Requirements of projects.
- Project team is small with prior experience.
- The problem is well understood and has been solved in the past.
- **Examples:** Simple Inventory Management Systems & Data Processing Systems.

Software Project Types

2. Semidetached Type:

- Medium size and has mixed rigid requirements. (50-300 KLOC)
- Project has Complexity not too high & low.
- Both experienced and inexperienced members in Project Team.
- Project are few known and few unknown modules.
- **Examples:** Database Management System, Difficult inventory management system.

3. Embedded Type:

- Large project with fixed requirements of resources. (More than 300 KLOC)
- Larger team size with little previous experience.
- Highest level of complexity, creativity and experience requirement.
- **Examples:** ATM, Air Traffic control, Banking software.



Types of COCOMO Model



Type 1: Basic COCOMO Model

- It is type of static model to estimates software development effort quickly and roughly.
- It mainly deals with the number of lines of code in project.

Formula:

$$\text{Effort (E)} = a \times (\text{KLOC})^b \text{ MM}$$

$$\text{Scheduled Time (D)} = c \times (E)^d \text{ Months (M)}$$

PROJECT TYPE	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Where,

- **E** = Total effort required for the project in Man-Months (MM).
- **D** = Total time required for project development in Months (M).
- **KLOC** = The size of the code for the project in Kilo lines of code.
- **a, b, c, d** = The constant parameters for a software project.

Type 1: Basic COCOMO Model Example

❑ Problem Statement:

- Consider a software project using semi-detached mode with 300 Kloc.
Find out Effort estimation, Development time and Person estimation.

❑ Solution:

$$\text{Effort (E)} = a \times (\text{KLOC})^b = 3.0 \times (300)^{1.12} = 1784.42 \text{ PM}$$

$$\text{Development Time (D)} = c \times (E)^d = 2.5 \times (1784.42)^{0.35} = 34.35 \text{ Months (M)}$$

$$\text{Person Required (P)} = \frac{E}{D} = \frac{1784.42}{34.35} = 51.9481 \text{ Persons} \approx 52 \text{ Persons}$$

PROJECT TYPE	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Type 2: Intermediate COCOMO Model

- Extension of Basic COCOMO model which enhance more accuracy to cost estimation model result.
- It include cost drivers (Product, Hardware, Resource & project Parameter) of project.

Formula:

$$\text{Effort (E)} = a * (\text{KLOC})^b * \text{EAF} \text{ MM}$$

$$\text{Scheduled Time (D)} = c * (\text{E})^d \text{ Months(M)}$$

Where,

- **E** = Total effort required for the project in Man-Months (MM).
- **D** = Total time required for project development in Months (M).
- **KLOC** = The size of the code for the project in Kilo lines of code.
- **a, b, c, d** = The constant parameters for the software project.

PROJECT TYPE	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

Type 2: Intermediate COCOMO Model

- **EAF: Effort Adjustment Factor**, which is calculated by multiplying the parameter values of different cost driver parameters. For ideal, the value is 1.

COST DRIVERS PARAMETERS	VERY LOW	LOW	NOMINAL	HIGH	VERY HIGH
Product Parameter					
Required Software	0.75	0.88	1	1.15	1.4
Size of Project Database	NA	0.94		1.08	1.16
Complexity of The Project	0.7	0.85		1.15	1.3

Hardware Parameter					
Performance Restriction	NA	NA	1	1.11	1.3
Memory Restriction	NA	NA		1.06	1.21
virtual Machine Environment	NA	0.87		1.15	1.3
Required Turnabout Time	NA	0.94		1.07	1.15

Personnel Parameter					
Analysis Capability	1.46	1.19	1	0.86	0.71
Application Experience	1.29	1.13		0.91	0.82
Software Engineer Capability	1.42	1.17		0.86	0.7
Virtual Machine Experience	1.21	1.1		0.9	NA
Programming Experience	1.14	1.07		0.95	NA

Project Parameter					
Software Engineering Methods	1.24	1.1	1	0.91	0.82
Use of Software Tools	1.24	1.1		0.91	0.83
Development Time	1.23	1.08		1.04	1.1

Type 2: Intermediate COCOMO Model Example

□ Problem Statement:

- For a given Semidetached project was estimated with a size of 300 KLOC. Calculate the Effort, Scheduled time for development by considering developer having ^{very} high application experience and very low experience in programming.

□ Solution:

$$EAF = 0.82 * 1.14 = 0.9348$$

$$\text{Effort (E)} = a * (\text{KLOC})^b * EAF = 3.0 * (300)^{1.12} * 0.9348 = 1668.07 \text{ MM}$$

$$\text{Scheduled Time (D)} = c * (E)^d = 2.5 * (1668.07)^{0.35} = 33.55 \text{ Months(M)}$$

PROJECT TYPE	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semidetached	3	1.12	2.5	0.35
Embedded	3.6	1.2	2.5	0.32

COST DRIVERS PARAMETERS	VERY LOW	LOW	NOMINAL	HIGH	VERY HIGH
Personnel Parameter					
Analysis Capability	1.46	1.19	1	0.86	0.71
Application Experience	1.29	1.13		0.91	0.82
Software Engineer Capability	1.42	1.17		0.86	0.7
Virtual Machine Experience	1.21	1.1		0.9	NA
Programming Experience	1.14	1.07		0.95	NA

Type 3: Detailed / Complete COCOMO Model

- The model incorporates all qualities of both Basic COCOMO and Intermediate COCOMO strategies on each software engineering process.
- The whole software is divided into different modules and then apply COCOMO in different modules to estimate effort and then sum the effort.

The Six phases of detailed COCOMO are:

1. Planning and requirements
 2. System design
 3. Detailed design
 4. Module code and test
 5. Integration and test
 6. Cost Constructive model
- 

Type 3: Detailed COCOMO Model Example

□ Problem Statement:

A distributed Management Information System (MIS) product for an organization having offices at several places across the country can have the following sub-components:

- Database part
- Graphical User Interface (GUI) part
- Communication part

□ Solution:

- ✓ The communication part can be considered as Embedded software.
- ✓ The database part could be Semi-detached software,
- ✓ The GUI part Organic software.

The costs for these three components can be estimated separately and summed up to give the overall cost of the system.

Advantages of COCOMO Model

1. Provides a systematic way to estimate the cost and effort of a software project.
2. Estimate cost and effort of software project at different stages of the development process.
3. Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. Provide ideas about historical projects.
5. Easy to implement with various factors.

Disadvantages of COCOMO Model

1. It ignores requirements, customer skills and hardware issues.
2. It limits the accuracy of the software costs.
3. It is based on assumptions and averages.
4. It mostly depends on time factors.
5. Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.

THE COCOMO II MODEL

- Stands for **CO**nstructive **CO**st **MO**del
- Introduced by **Barry Boehm** in 1981
- Became one of the **well-known** and **widely-used** estimation models in the industry
- It has evolved into a more **comprehensive** estimation model called **COCOMO II**, with reuse property.

THE COCOMO II MODEL

- COCOMO II is actually a 3 level hierarchy of estimation models that address the following areas:
 - Application composition model - Used during the early stages of software engineering process.
 - Early design stage model - Used once requirements have been stabilized and basic software architecture has been established.
 - Post-architecture-stage model - Used during the construction of the software.

THE COCOMO II MODEL

- The COCOMO II models require sizing information
- Three different sizing options are available as part of the model hierarchy:
 - Object points (OP)
 - Function points (FP)
 - Lines of source code (LOC)
- The COCOMO II application model uses object points (OP)

THE COCOMO II MODEL

- Object Point is an indirect software measure that is computed using counts of no. of –
 - Screens
 - Reports
 - Components likely to be required to build the application
- Each of the above object instance is classified into one of the three complexity levels – simple, medium or difficult

OBJECT POINTS TABLE

Object Type	Simple	Meduim	Difficult
Screen	1	2	3
Report	2	5	8
Each 3GL module	10	10	10

THE COCOMO II MODEL

- The object point count is then determined by multiplying the original no. of object instances by the weighting factor and summing to obtain a total object point count

$$\text{New Object Point (NOP)} = (\text{object points}) * [(100 - \%reuse)/100]$$

To derive an estimate of effort based on the computed NOP value, a “**productivity rate**” must be derived.

Productivity rate, PROD = OP / person-month

Once the productivity rate has been determined, an **estimate of project effort** is computed using

$$\text{Estimated Effort} = \text{NOP} / \text{PROD}$$

PRODUCTIVITY METRIC TABLE

Developer's experience and Capability / ICASE maturity and capability	Very low	Low	Nominal	High	Very high
PROD: Productivity Object-point per person- month	4	7	13	25	50

Object Point Estimation Procedure

1. Assess **object-counts** in the system: number of screens, reports, & 3GL.
2. Assess complexity level for each object (use table): simple, medium and difficult.
3. Calculate **"NOP" the object-point count of the system**: add all weights for all object instances
4. Estimate the % of reuse and compute the adjusted NOP **"New Object Points"** to be developed
5. Determine the productivity rate PROD (use metrics table)
6. Compute the adjusted effort **$PM = \text{adjusted NOP} / \text{PROD}$**

Example

The system includes:

- 6 screens: 2 simple + 3 medium + 1 difficult
- 3 reports: 2 medium + 1 difficult
- 2 3GL components
- 30 % of the objects could be supplied from previously developed components
- Productivity is high.

Calculate estimated effort.

Solution

Object counts:

2 simple screens	x	1	=	2
3 medium screens	x	2	=	6
1 difficult screen	x	3	=	3
2 medium reports	x	5	=	10
1 difficult report	x	8	=	8
2 3GL components	x	10	=	20

OP Count 49

Object Type	Simple	Meduim	Difficult
Screen	1	2	3
Report	2	5	8
Each 3GL module	10	10	10

Solution

1. New Object Point (NOP) = (object points) * [(100 - %reuse)/100]

$$\begin{aligned} &= (49) * [(100 - 30)/100] \\ &= 34.3 \\ &= 35 \end{aligned}$$

2. Productivity rate, PROD= OP/ person-month

For High Productivity (Metric Table) : PROD = 25 OP/ person-month

3. Estimated Effort = NOP / PROD

$$= 35/25 = 1.4 \text{ person-month}$$